Consultas — Etapa 4 (chequeo de sentencias)

Agustín Paul — L.U. 147524

Te paso las dudas que me surgieron con la Etapa 4. Las dos últimas son sólo para confirmar. En cada una hay un ejemplo, lo que da hoy mi compilador y la alternativa. Con un «sí» o con la otra opción ya me alcanza. Cuando cito «el documento» me refiero a *Semántica de MiniJava – Documento completo* (v3.0), y «la Parte I» es el documento de la Etapa 3. Las líneas de los ejemplos arrancan en 1.

1. ¿Los operadores binarios tienen precedencia, o van todos en un mismo nivel?

```
1 class Aa {
2     void m(int x, int y) {
3         var b = x < y + 1;
4     }
5 }
```

Hoy da `[Error:+|3]`: pongo los trece operadores en un mismo nivel y asocio a izquierda, así que queda `(x < y) + 1`, y `boolean + int` no tipa. Lo hice así por el §11, 2.º párrafo: «El chequeo semántico no introduce la precedencia de Java ni reorganiza operadores [...] la política de desambiguación es la fijada para Etapa 2».

2. Uno de los casos que mandaste usa ++, que es de un logro. ¿Es un error del caso?

`semIICorrecto03.java`, de `testersSemantico2.zip`, espera `[SinErrores]` y tiene:

```
1 class A1 {
2     int a1;
3     int a2;
4     void m1(){
5         a2 = a1++ + 10;
6     }
7 }
```

El `++` posfijo viene del logro Operadores Posfijos de la Etapa 2, y controlarlo es el logro «Postfijos – Chequeado» (Pautas, slide 48), que no persigo. Como me dijiste que los programas de prueba de una etapa no traen construcciones de logros que no se persiguen, no lo controlo, y este caso da error.
¿Lo tomo como un descuido del caso, o la batería de la cátedra usa posfijos?

3. ¿Un genérico sin argumento (raw) es error también en atributos, parámetros y retornos?

```
class Caja<T> { }
class Aa { Caja x; }
```

Hoy da `[Error:Caja|2]`, siguiendo el §5.2 del documento: «Fuera de las cláusulas de herencia o implementación, usar una entidad genérica sin argumento es un error semántico. Esto incluye atributos, parámetros, retornos, tipos base de arreglos y creaciones de objetos». Pero la Parte I, §3, lo permitía («Puede tener un argumento genérico opcional: Caja, Caja<T> o Caja<String>»), y con eso entregué la Etapa 3.
Alternativa: `[SinErrores]`, y el raw sólo es error en los cuerpos (`new Caja()`).

4. Dos parámetros de tipo con el mismo nombre, ¿son el mismo tipo?

```
class Base<T> { T get() { return null; } }
class Der<T> extends Base { T get() { return null; } }
```

`Der` hereda sin argumento, así que conserva la `T` de `Base`, y además declara la suya. Hoy da `[Error:get|2]`: las tomo como tipos distintos y la redefinición no tiene el mismo retorno. Me baso en el §5.1, último párrafo: «Para representar los tipos se debe conservar su declaración de origen, además del nombre».
Alternativa: `[SinErrores]`, si dos `T` con el mismo nombre son el mismo tipo.

5. ¿Se puede llamar a `toString()` sobre algo de tipo interfaz?

```
interface Ix { }
class Aa { void m(Ix i) { i.toString(); } }
```

Hoy da `[SinErrores]`, como en Java, donde una interfaz tiene los métodos públicos de `Object`. El §10.2 dice que «Las interfaces permiten seleccionar sus métodos declarados y heredados», y no menciona los de `Object`.
Alternativa: `[Error:toString|2]`.

6. Si un argumento no conforma con su parámetro, ¿qué token reporto?

```
class Aa {
    Aa(int x) { }
    void f(int x) { }
    void m() {
        f(true);
        var a = new Aa(true);
    }
}
```

En el método uso el nombre, como en tu caso `semIIError04` (`[Error:m2|10]`): da `[Error:f|5]`, y lo mismo con `a.f(true)` o `Aa.f(true)`. En el constructor no encontré un criterio en el material, así que sigo a `javac`, que marca el argumento: `[Error:true|6]`.
¿Está bien? Alternativas para el constructor: `[Error:Aa|6]` o `[Error:new|6]`.

7. Para confirmar: en una asignación, ¿se chequean los dos lados antes de ver si el izquierdo es asignable?

```
1 class Aa {
2     void m() {
3         this = x;
4     }
5 }
```

Hoy da `[Error:x|3]`: sigo el pseudocódigo de `NodoExpAsig` de la Clase 11 (slides 26, 31 y 36), que chequea los dos lados, después la conformidad y al final la asignabilidad. `javac` lo hace al revés y marca primero que `this` no se puede asignar.
Alternativa: `[Error:=|3]`.

8. Para confirmar: en una sentencia que no es asignación ni llamada, ¿se chequean los tipos antes que la forma?

```
1 class Aa {
2     void m() {
3         true + zz;
4     }
5 }
```

Hoy da `[Error:zz|3]`, por el §11, 1.er párrafo: «Cada expresión chequea sus subexpresiones antes de determinar su resultado». `javac` marca primero que no es una sentencia válida.
Alternativa: `[Error:+|3]`.